

Hardware & Architecture Decisions

Betsybots · DIY Challenge 2026 · Team Discussion Document

May 21, 2026 — Capturing decisions from team discussion

This document captures the technical discussions and recommendations around sensor choices, compute architecture, and localization strategy. Each section presents the question raised, the analysis, and the team recommendation.

1. RTK GPS — Does It "Defeat the Purpose" of Localization?

The Concern

Team member raised: "Using an accurate RTK GPS module with 1 cm accuracy defeats the purpose of building a localization system. If GPS is that good, why do we need anything else?"

Why This Concern is Partially Valid but Incomplete

- **GPS gives position only** — no rotation/heading. You still need IMU + LiDAR for orientation.
- **GPS drops out** — under trees, near buildings, in tunnels, or if the competition has GPS-denied zones.
- **GPS rate is slow** — 10–20 Hz vs LiDAR odometry at 100+ Hz. Real-time control needs the fast rate.
- **GPS alone can't navigate** — you need relative odometry for obstacle avoidance and path tracking.

The real architecture: **FAST-LIO gives primary odometry, RTK is a drift-correction signal in the EKF.** If GPS drops, the system still works. If GPS is there, it prevents long-term drift accumulation.

Alternatives Discussed

Option	Pros	Cons
RTK GPS (secondary EKF input)	Drift-free position Cheap modules (\$150–200) Self-contained (just needs sky)	No rotation data Drops in some areas Slow update rate
Dead-wheel encoders	Simple, reliable, fast rate \$20–30 total cost Gives distance + heading (differentiable)	Drifts on rough terrain (slip) Needs calibration Mechanical wear
Visual odometry (stereo/mono camera)	Works indoors No external infrastructure	Compute-heavy Fragile outdoors (lighting) Needs GPU or heavy CPU

✓ Use RTK as secondary EKF input (not primary odom). Add dead-wheel encoders as cheap insurance. Skip visual odometry.

Why Dead-Wheel Encoders Are Worth Adding

- \$20–30 total cost — trivial compared to other sensors
- Give you wheel-slip detection (compare encoder odom vs LiDAR odom)
- Fast update rate (1000+ Hz) — helps EKF between LiDAR scans
- Easy ROS 2 integration — just publish nav_msgs/Odometry
- Works as a fallback if both GPS and LiDAR have issues simultaneously

2. UWB (Ultra-Wideband) Positioning

The Question

"Can we use UWB for positioning? Is it allowed by the competition rules?"

Analysis

- UWB requires **pre-placed anchors** around the course at known positions
- If the competition organizers don't provide anchors and you can't place your own → not practical
- RTK GPS is self-contained — just needs sky view, no external infrastructure
- UWB gives ~10 cm accuracy indoors (comparable to RTK outdoors), but adds infrastructure dependency

✓ Check competition rules. If allowed AND you can place your own anchors, it's viable. Otherwise, RTK is simpler and self-contained.

Action item: Review the official competition rules document for permitted positioning systems and any restrictions on pre-placed infrastructure.

3. AprilTag Landmarks for Online Position Correction

The Idea

During offline map generation (without obstacles), place AprilTags at known positions. During competition, camera detects these tags and provides global position corrections to the EKF.

How This Works

- **Step 1:** During offline mapping run, place AprilTags at key positions on the course
- **Step 2:** Survey each tag's global position (using RTK or manual measurement)
- **Step 3:** Record tag positions in the map as known landmarks
- **Step 4:** During competition, camera detects tags → known global position → feeds EKF
- **Result:** Acts like "indoor GPS" — gives global position fixes at specific points

Where to Place Tags

Location	Why	Priority
Start/finish line	Known exact position, visible on every lap	High
Before obstacle zones	Reset position error before critical navigation	High
After obstacle zones	Correct any drift accumulated during obstacle	Medium
At major turns	Heading correction at geometry changes	Medium
Along straightaways	Drift correction on long sections	Low

Advantages of This Approach

- Works even WITHOUT RTK GPS — tags give you absolute position
- CPU-lightweight: AprilTag detection runs at ~30 Hz on ARM without GPU
- Zero cost: just printed paper tags on rigid boards
- Can be combined with RTK — tag corrections fill GPS gaps (near structures, etc.)
- Already planned: our AprilTag calibration guide covers the EKF integration

✓ **YES — implement AprilTag landmarks. Place tags during offline mapping, survey positions with RTK, use for online EKF corrections.**

4. Our IMU: ACEINNA MTLT335D

What We Have

The ACEINNA MTLT335D is an industrial-grade 6-DOF IMU — significantly better than consumer-grade units like the RealSense D435i's built-in BMI055 or typical \$15 breakout boards.

Specification	MTLT335D (Ours)	Typical Consumer IMU
DOF	6 (3-axis accel + 3-axis gyro)	6 or 9
Gyro bias stability	1.3 °/hr (excellent)	10–50 °/hr
Roll/Pitch accuracy	0.05° (built-in KF)	0.5–2°
Gyro range	±400 °/s	±250–2000 °/s
Accel range	±8 g	±2–16 g
Output rate	100/200 Hz	100–1000 Hz
Redundancy	Triple-redundant with fault detection	Single sensor
Temp calibration	Full range -40°C to +85°C	None or limited
Interface	CAN 2.0 / RS232	I2C / SPI / USB
Built-in processing	16-state EKF (pitch/roll)	None
Housing	IP67 rugged sealed	Bare PCB

6-DOF vs 9-DOF — Do We Need a Magnetometer?

The MTLT335D is 6-DOF (no magnetometer). This is **fine** for our application:

- FAST-LIO does NOT use magnetometer data — only gyro + accel
- Heading comes from LiDAR scan matching (very accurate) + gyro integration (fast)
- Outdoor magnetometers are unreliable near motors, batteries, and metal structures
- Our robot starts at a known heading; LiDAR-to-map matching gives absolute heading within seconds

✓ 6-DOF is correct for this application. Do NOT add a magnetometer.

Integration Requirements

The MTLT335D outputs on **CAN 2.0 or RS232**, not USB. Integration path:

Integration Steps

- Option A: CAN-to-USB adapter (PEAK PCAN-USB ~\$40) connected to Orin Nano
- Option B: RS232 output with USB-to-serial adapter

- Write a thin ROS 2 driver node that parses MTLT335D protocol → publishes sensor_msgs/Imu
- Protocol is documented in ACEINNA user manual (available on their site)
- Mount RIGIDLY to LiDAR frame — vibration isolation between them ruins calibration

■ **Do NOT use the RealSense D435i built-in IMU. The MTLT335D is vastly superior. The D435i IMU is consumer-grade and poorly positioned relative to the LiDAR.**

5. Camera: RealSense D435i vs D555

Current Role of the Camera

The camera is a **secondary/helper sensor** — not on the critical path. If it dies mid-run, the robot still navigates. Three use cases:

Use Case	What It Does	Priority
AprilTag detection	EKF position corrections at known landmarks (CPU, RGB only)	Medium
Visual start signal	Watch for "go" signal to avoid 5-sec penalty	Low
Depth assist (chimes)	One specific obstacle where LiDAR has ground-level blind spot	Low

D435i vs D555 Comparison

Feature	D435i (current)	D555 (available)
Outdoor depth range	~3 m usable in sunlight	~6 m in sunlight
Ambient light rejection	Basic	Significantly better
RGB quality	Good (1080p)	Good (1080p)
Built-in IMU	Yes (BMI055)	No
ROS 2 driver	realsense2_camera	Same — drop-in replacement
Form factor	Small, USB-C	Similar, USB-C
Cost	Already have	Already available

✓ Either camera works. D555 is a free upgrade if available (better outdoor depth). Since we use the MTLT335D as our IMU, losing the D435i's built-in IMU is irrelevant.

Low priority — swap only after core system is working. The camera is the least critical sensor.

6. Compute Architecture — Orin Nano vs Multiple Pis

The Question

"Is it better to use multiple Raspberry Pis, or stick with Orin Nano + one Pi?"

Architecture Comparison

	Orin Nano + 1 Pi	3× Raspberry Pi 5
Total CPU cores	6 (Orin A78AE) + 4 (Pi A76) = 10	3×4 = 12 (but weaker per-core)
Single-core perf	Orin ~2× faster per-core than Pi 5	Slower per-core

RAM	8 GB + 8 GB = 16 GB (shared bus)	3×8 = 24 GB (split across machines)
GPU	40 TOPS (available if needed)	None
Network overhead	1 hop (DDS between 2 machines)	2–3 hops, 3-way DDS discovery
Time sync	1 pair (PTP/chrony)	3-way sync (harder)
Power draw	~15W + ~5W = 20W	3×8W = 24W
Debug complexity	Low — one main brain	High — which Pi has what?

Why Multiple Pis Is a Bad Fit

- FAST-LIO needs sustained single-core performance at 100 Hz — Pi 5 barely keeps up
- Nav2 planner/controller is memory-hungry and CPU-intensive — wants Orin's fast cores
- Splitting ROS 2 nodes across Pis = network latency on /cmd_vel, /scan, /odom
- Debugging multi-machine ROS 2: which machine's log? which DDS domain? clock drift?
- No team member should spend time on distributed systems debugging instead of the actual challenge

When to Add the Single Pi

- Only if CPU contention is observed during full-stack testing (check with htop)
- Offload camera node (realsense + AprilTag) to Pi to free Orin CPU
- Offload bag recording (disk I/O on separate device)

✓ Orin Nano runs everything. Add 1 Pi ONLY if htop shows CPU contention during full runs. Never use multiple Pis.

7. GPU Utilization — Stretch Goals (July/August)

The Orin Nano has **1024 CUDA cores / 40 TOPS** sitting completely idle in the current stack. Everything we run (FAST-LIO, Nav2, EKF) is CPU-only. If we want extra robustness after the base system works, here's what the GPU can do — all from NVIDIA's Isaac ROS packages.

Isaac ROS Packages (Pre-built for Jetson + ROS 2 Humble)

Package	What It Does	Benefit to Us
Isaac ROS cuVSLAM	GPU-accelerated stereo visual SLAM from camera images	Second odometry source Runs on GPU = zero CPU cost Works when LiDAR struggles
Isaac ROS Nvblox	Real-time 3D voxel map from depth + LiDAR on GPU	True 3D obstacle awareness Catches overhanging obstacles Better than 2D costmap
Isaac ROS AprilTag	GPU-accelerated tag detection	Faster/more reliable than CPU apriltag_ros Frees CPU for other tasks
Isaac ROS Freespace Segmentation	DNN classifies drivable surface vs obstacle	Could replace LiDAR cluster classifier But needs training data — lower priority

Highest-Value Addition: cuVSLAM

GPU-accelerated Visual SLAM from NVIDIA, purpose-built for Jetson. Takes stereo images (from D435i or D555) and outputs visual odometry at 60+ Hz — entirely on GPU.

- **Runs on GPU** → CPU stays free for FAST-LIO + Nav2
- **Second independent odometry source** → feeds EKF as Odom1
- **Works when LiDAR might struggle** — featureless open field, dust, rain
- **Zero additional hardware** — uses the camera already on the robot
- **Pre-built ROS 2 package** — apt install ros-humble-isaac-ros-visual-slam
- **Integration effort** — 2–3 hours (launch file + EKF config addition)

EKF Architecture With cuVSLAM Added

Source	Sensor	Output	Runs On	Rate
FAST-LIO	QT64 + MTLT335D	Odom0 (position + orientation)	CPU	100 Hz
cuVSLAM	D435i/D555 stereo	Odom1 (visual odometry)	GPU	60 Hz
MTLT335D	IMU direct	IMU0 (angular velocity)	CPU	200 Hz
Wheel encoders	Dead-wheels	Odom2 (wheel odometry)	CPU	100+ Hz
RTK GPS	GPS module	GPS0 (global position)	CPU	10–20 Hz
AprilTags	Camera RGB	Pose corrections (event)	GPU*	30 Hz

* Isaac ROS AprilTag runs on GPU; standard apriltag_ros runs on CPU.

Key insight: Two independent odometry sources (LiDAR + visual) means if either has a bad moment, the EKF detects the disagreement and weights the good one higher. This is significantly more robust than any single-source approach.

Nvblox — 3D Costmap on GPU

Nav2's default costmap is 2D (single height slice). Nvblox builds a full 3D voxel map on GPU from depth camera + LiDAR point cloud, then outputs a 2D costmap slice for Nav2.

- Catches **overhanging obstacles** (branches, signs) that 2D LiDAR misses
- Better for **chimes obstacle** (thin vertical objects at various heights)
- **GPU-accelerated** → no CPU cost
- Outputs standard nav_msgs/OccupancyGrid → plugs directly into Nav2

What NOT to Use the GPU For

- **YOLO / object detection** — not needed, LiDAR geometry is sufficient for our obstacles
- **Semantic segmentation** — overkill, we don't need pixel-level classification
- **Any model requiring training data** — we don't have labelled datasets and can't create them in time
- **Anything foundational** — GPU features are stretch goals, not critical path

Timeline for GPU Integration

- June: DO NOT touch GPU features — focus on calibration, mapping, base stack
- July (if base system is stable): Add cuVSLAM as Odom1 in EKF — 2–3 hour task
- Aug 1–15 (if cuVSLAM works well): Consider Nvblox for 3D costmap
- Aug 15: HARD FEATURE LOCK — 45 days of bug fixes and practice runs until Oct 1

✓ GPU stretch goals are additive, not foundational. Get the CPU-only stack working first. cuVSLAM in July is the highest-value GPU addition.

8. \$500 Budget Allocation

Recommended Spend

Item	Est. Cost	Why
RTK GPS module (SparkFun / ArduSimple)	\$150–200	Drift-free position corrections in EKF
Dead-wheel encoder kit	\$20–30	Cheap odom backup, slip detection
CAN-to-USB adapter (for MTLT335D)	\$40	Connect IMU to Orin Nano
Extra LiPo batteries	\$60–80	Extended field testing sessions
Connectors, cables, mounting hardware	\$40–50	Rigid sensor mounting, wiring
AprilTag boards (rigid printing)	\$15–20	Printed tags on foam/aluminum board
Buffer for breakage	\$100–150	Spare parts, replacement cables, etc.

Total estimated: \$425–530 — within budget with buffer for unexpected needs.

What NOT to Spend On

- **Additional compute boards** — Orin Nano is sufficient
- **Better camera** — D555 already available, D435i already sufficient
- **Better IMU** — MTLT335D is industrial-grade, more than enough
- **Magnetometer** — not needed, unreliable near motors
- **UWB system** — needs infrastructure, uncertain if allowed
- **GPU inference hardware** — we're not doing camera-based detection

9. Final Sensor Architecture Summary

Sensor	Role	EKF Layer	Priority
Hesai QT64 LiDAR	Primary odometry (via FAST-LIO) Obstacle detection (point cloud)	Odom0 (100 Hz)	CRITICAL
ACEINNA MTLT335D	Gyro + accel for FAST-LIO Angular velocity for EKF	IMU0 (200 Hz)	CRITICAL
RTK GPS (new)	Global position correction Prevents long-term drift	GPS0 (10–20 Hz)	HIGH
Dead-wheel encoders (new)	Wheel odometry Slip detection	Odom1 (100+ Hz)	MEDIUM

Camera (D435i/D555)	AprilTag detection Start signal Chimes depth assist	Via AprilTag (event-based)	LOW
---------------------	---	-------------------------------	-----

EKF Fusion Architecture

Layer	Inputs	Output	Rate
FAST-LIO (LiDAR-inertial)	QT64 point cloud + MTLT335D IMU	High-rate odometry (position + orientation)	100 Hz
Local EKF (robot_localization)	FAST-LIO odom + wheel encoders + IMU angular velocity	Smooth local odom (for Nav2 control)	50 Hz
Global EKF (robot_localization)	Local odom + RTK GPS + AprilTag detections	Global pose estimate (map frame)	50 Hz

Key insight: Each layer adds robustness. If any single sensor fails, the other layers continue working. RTK drops? Local EKF + LiDAR still navigates. Wheel slip? LiDAR odom is unaffected. Camera obscured? Only AprilTag corrections stop — everything else is fine.

10. Decisions Summary

#	Question	Decision	Status
1	RTK GPS defeats localization?	No — use as secondary EKF correction, not primary	DECIDED
2	Dead-wheel encoders?	Yes — add them (\$20, easy integration)	DECIDED
3	Visual odometry?	No — too compute-heavy, fragile outdoors	DECIDED
4	UWB allowed?	Check rules. Probably skip (needs infrastructure)	PENDING
5	AprilTag landmarks?	Yes — place during offline map, use for online correction	DECIDED
6	Magnetometer needed?	No — 6-DOF is correct, mag unreliable near robots	DECIDED
7	IMU choice?	ACEINNA MTLT335D — industrial grade, already have it	DECIDED
8	Camera D435i vs D555?	Either works. Swap to D555 if convenient, low priority	DECIDED
9	Multiple Pis?	No — Orin Nano handles everything. Add 1 if needed	DECIDED
10	Separate gyroscope?	No — MTLT335D has excellent gyro (1.3°/h stability)	DECIDED
11	GPU utilization?	cuVSLAM in July as stretch goal. Nvblox in Aug. None before.	PLANNED

Action Items for Next Session

- Check competition rules for UWB / external infrastructure restrictions
- Source RTK GPS module (SparkFun RTK Express or ArduSimple)
- Source dead-wheel encoder kit (rotary encoders + mounting brackets)
- Get CAN-to-USB adapter for MTLT335D integration
- Write MTLT335D ROS 2 driver node (parse CAN/serial → sensor_msgs/Imu)
- Plan AprilTag placement during first offline mapping run